

Installation

pip install pandas opencv-python numpy

Generate synthetic dataset:

1. A **2-minute MP4 video** (10 FPS = 1200 frames) simulating a scope moving/expanding.
2. A correlated **CSV file** containing **timestamp**, **flow_rate**, and **pressure**.

```
import cv2
```

```
import numpy as np
```

```
import pandas as pd
```

```
# Parameters
```

```
fps = 10
```

```
duration_sec = 120 # 2 minutes
```

```
total_frames = fps * duration_sec
```

```
width, height = 320, 240
```

```
# 1. Generate Timestamps & Synthetic Sensor Signals
```

```
timestamps = np.linspace(0, duration_sec, total_frames)
```

```
# Synthetic pressure (sine wave + noise) & flow rate (proportional response)
```

```
pressure = 20 + 10 * np.sin(2 * np.pi * 0.05 * timestamps) + np.random.normal(0, 0.5,  
total_frames)
```

```
flow_rate = 0.5 * pressure + np.random.normal(0, 0.2, total_frames)
```

```
# Save CSV
```

```
df = pd.DataFrame({  
    'timestamp_sec': timestamps,  
    'pressure_psi': pressure,  
    'flow_rate_lpm': flow_rate  
})
```

```
df.to_csv('sensor_data.csv', index=False)
```

```
print("Saved sensor_data.csv successfully!")
```

```
# 2. Generate Synthetic Video (Circle expands/shrinks with pressure)
```

```
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
```

```
out = cv2.VideoWriter('scope_video.mp4', fourcc, fps, (width, height))
```

```
for p in pressure:
```

```
    # Blank frame (black background)
```

```
    frame = np.zeros((height, width, 3), dtype=np.uint8)
```

```
    # Circle radius scales with pressure signal
```

```
    radius = int(np.clip(p * 3, 10, 100))
```

```

cv2.circle(frame, (width // 2, height // 2), radius, (0, 255, 128), -1)

out.write(frame)

out.release()
print("Saved scope_video.mp4 successfully!")

```

2. A correlated **CSV file** containing **timestamp**, **flow_rate**, and **pressure**.

Establish Data Organizing Framework

To train a model later (e.g., PyTorch or TensorFlow), your inputs (X: flow/pressure) and outputs (Y: video frames) must be aligned by time.

```

import cv2
import pandas as pd

def load_and_align_dataset(csv_path, video_path):
    # Load sensor data
    sensor_df = pd.read_csv(csv_path)

    # Load video frames
    cap = cv2.VideoCapture(video_path)
    frames = []

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        frames.append(frame)
    cap.release()

    # Truncate to match minimum length in case of minor frame/timestamp mismatch
    min_len = min(len(sensor_df), len(frames))

    X = sensor_df[['flow_rate_lpm', 'pressure_psi']].iloc[:min_len].values
    Y = np.array(frames[:min_len])

    return X, Y

# Test alignment
X_train, Y_train = load_and_align_dataset('sensor_data.csv', 'scope_video.mp4')
print(f"Inputs (Flow, Pressure) Shape: {X_train.shape}") # Shape: (1200, 2)
print(f"Outputs (Video Frames) Shape: {Y_train.shape}") # Shape: (1200, 240, 320, 3)

```

Choosing architecture

A lightweight Multi-Layer Perceptron (MLP) or Convolutional Transpose Network (DeconvNet) mapping (Flow, Pressure) directly to an image pixel array.

Install Pytorch

```
pip install torch torchvision
```

Build and train the model (in notepad)

```
import cv2
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# 1. Function to Load and Align Dataset
def load_and_align_dataset(csv_path, video_path):
    sensor_df = pd.read_csv(csv_path)
    cap = cv2.VideoCapture(video_path)
    frames = []

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        frames.append(frame)
    cap.release()

    min_len = min(len(sensor_df), len(frames))
    X = sensor_df[['flow_rate_lpm', 'pressure_psi']].iloc[:min_len].values
    Y = np.array(frames[:min_len])

    return X, Y

# 2. Load and Prepare Tensors
X, Y = load_and_align_dataset('sensor_data.csv', 'scope_video.mp4')

# Normalize inputs and outputs
X_norm = (X - X.mean(axis=0)) / X.std(axis=0)
Y_norm = np.transpose(Y, (0, 3, 1, 2)) / 255.0

X_tensor = torch.tensor(X_norm, dtype=torch.float32)
```

```
Y_tensor = torch.tensor(Y_norm, dtype=torch.float32)
```

```
dataset = TensorDataset(X_tensor, Y_tensor)
```

```
dataloader = DataLoader(dataset, batch_size=16, shuffle=True)
```

3. Define Generator Model

```
class ScopelImageGenerator(nn.Module):
```

```
    def __init__(self):
```

```
        super(ScopelImageGenerator, self).__init__()
```

```
        self.fc = nn.Sequential(
```

```
            nn.Linear(2, 256),
```

```
            nn.ReLU(),
```

```
            nn.Linear(256, 128 * 15 * 20),
```

```
            nn.ReLU()
```

```
        )
```

```
        self.deconv = nn.Sequential(
```

```
            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2, padding=1),
```

```
            nn.BatchNorm2d(64),
```

```
            nn.ReLU(),
```

```
            nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1),
```

```
            nn.BatchNorm2d(32),
```

```
            nn.ReLU(),
```

```
            nn.ConvTranspose2d(32, 16, kernel_size=4, stride=2, padding=1),
```

```
            nn.BatchNorm2d(16),
```

```
            nn.ReLU(),
```

```
            nn.ConvTranspose2d(16, 3, kernel_size=4, stride=2, padding=1),
```

```
            nn.Sigmoid()
```

```
        )
```

```
    def forward(self, x):
```

```
        out = self.fc(x)
```

```
        out = out.view(-1, 128, 15, 20)
```

```
        out = self.deconv(out)
```

```
        return out
```

4. Train Model

```
model = ScopelImageGenerator()
```

```
criterion = nn.MSELoss()
```

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
print("🚀 Starting Training Loop...\n")
```

```
for epoch in range(10):
```

```
    total_loss = 0.0
```

```
    for batch_x, batch_y in dataloader:
```

```
optimizer.zero_grad()
predictions = model(batch_x)
loss = criterion(predictions, batch_y)
loss.backward()
optimizer.step()
total_loss += loss.item()
```

```
print(f'Epoch [{epoch+1}/10] - Loss: {total_loss / len(dataloader):.6f}')
```

```
print("\n✅ Training Complete!")
```

5. Generate and Save Test Image

```
model.eval()
with torch.no_grad():
    sample_input = X_tensor[0].unsqueeze(0)
    predicted_frame = model(sample_input).squeeze(0).numpy()
    predicted_frame = (np.transpose(predicted_frame, (1, 2, 0)) * 255.0).astype(np.uint8)
    cv2.imwrite('predicted_scope_frame.png', predicted_frame)
    print("Saved predicted frame as 'predicted_scope_frame.png'")
```

Paste saved notebook into windows command

```
python train_model.py
```

Visualize training loop

Input Telemetry: Flow rate & pressure values are read and normalized.

Feature Mapping: The neural network takes those 2 values and projects them into spatial feature maps.

Image Synthesis: The deconvolutional layers reconstruct a 2D synthetic scope frame (240 × 320 × 3).

Loss Minimization: Pixel-level error decreases with every batch iteration.

Comparing Results - write in notepad

```
import cv2
import numpy as np
```

```
# Load original ground truth frame and neural network prediction
ground_truth = cv2.imread('predicted_scope_frame.png') # Predicted output from
train_model.py
cap = cv2.VideoCapture('scope_video.mp4')
ret, original_frame = cap.read()
cap.release()
```

```
if ret and ground_truth is not None:
    # Add text labels
```

```

cv2.putText(original_frame, "Original Ground Truth", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
cv2.putText(ground_truth, "Model Synthetic Frame", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

# Combine side-by-side into a single image
comparison = np.hstack((original_frame, ground_truth))

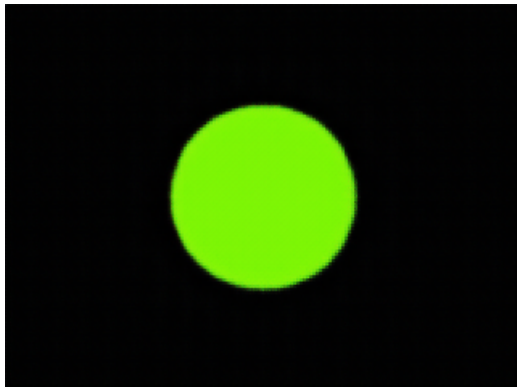
# Save comparison image
cv2.imwrite('side_by_side_comparison.png', comparison)
print("Saved 'side_by_side_comparison.png'! Open this file to inspect your model's
prediction.")
else:
    print("Could not load frames. Make sure train_model.py finished successfully.")

```

Paste into Windows command

```
python compare_results.py
```

Predicted Scope Image - Synthetically Generated



Compared to the frame from the video



HOW THIS WORKS

1. Input Features (X) and Output Features (Y)
 - a. X = flow rate and pressure measurements at a specific moment in time (a 2-element vector)
 - b. Y = actual 2D camera image from the video corresponding to the exact timestamp (240 x 320 x 3 pixels = 230,400 individual RGB numbers)
2. Model Architecture (Learnable Function)
 - a. Scope generator is a convolutional neural network - specifically a deconvolutional/decoder network
 - i. Starts with 2 input values
 - ii. Passes them through linear layers (nn.Linear) to expand 2 values into high dimensional numerical feature maps
 - iii. Uses transpose convolutions (nn.ConvTranpose2d) to progressively scale spatial dimensions from tiny grid up to full video resolution
 - iv. Contains thousands of trainable parameters (weights and biases) that initially contain random numbers
3. Loss and Backwards Propagation (Learning Process)
 - a. Forward pass = takes flow and pressure, computes synthetic image and output its best guess
 - b. Loss Function (nn.MSELoss) = compares generated image against actual ground-truth frame pixel by pixel. Mean squared error measures how far off generated pixels are from reality
 - c. Backpropagation (loss.backward() & optimizer.step()) Pytorch calculates gradients to adjust the internal weights slightly so the next calculation has lower error. Model actively learns the relationship between pressure and visual shape.

HOW TO USE WITH REAL DATA

1. load_and_align_dataset command handles temporal alignment (X and Y sharing matching frames indices) which remains identical no matter the amount of frames present
2. Established input interface and output interface - when sponsor data arrives
 - a. Data loaders can stay the same
 - b. Training loop stays the same
 - c. Only need to upgrade the model inside ScopeImageGenerator (swapping baseline CNN to Conditional GAN or Diffusion Model to generate photo-realistic tissue textures)
3. Immediate baseline marker
 - a. Need something to compare synthetic image against
 - b. This model provides a very simple baseline speed and accuracy benchmark
 - c. Testing complex models later, can measure whether added complexity produces better results than initial baseline

April Summary:

1. Asked Gemini to create a 2 minute long video of a changing size circle that changes based on “pressure” and “flow” variables
 - a. Not actual flow and pressure data, just using as variable names for the project and to describe relationship in model
2. Asked Gemini to then create synthetic data under the variable “pressure” and “flow” that correlate to the relationship of the circle changing shape.
3. Gemini was then asked to use the pressure and flow data it created to analyze their relationship to each other to create a synthetic image that should be identical to the image in the video it created.
4. The model ran a temporal model to create elements/vectors using the pressure and flow data and then studied them to determine the relationship between the variables and the size of the circle at a given frame.
5. ML generated an image based on what it learned from the data relationship and it was compared to the original frame/image from the video the variables were based off.
 - a. Images were identical (shape and size)
6. Overall goal of model: given a specified pressure and flow data, it could use the relationship between them to create a circle (or other image) similar to the image seen in the actual video.